

FDA – NEU

Data Structures and Algorithms in Python
Lecture 4: Sorting, Selection and Searching Algorithms

Faculty of Data Science and Artificial Intelligence

National Economics University

Lecture 4, Part 2: Table of Contents

1 Selection

2 Searching

Learning outcomes

By the end of this lecture, students should be able to:

- ① Distinguish selection from complete sorting.
- ② Find extremes, order statistics, and top- k values efficiently.
- ③ Explain when sorting, quickselect, or a heap is appropriate.
- ④ Implement linear, binary, and ternary search in pseudocode.
- ⑤ Choose a search method based on data organization and workload.

SECTION 1

Selection

Selection asks for part of the order

Selection problem

Given n values, find the record with rank k without necessarily sorting every record.

Examples

- Minimum or maximum
- Median
- Third-smallest value
- Top ten scores

Key question

If only one rank is required, is the work of completely sorting the input necessary?

Rank and order statistics

Sorted view of the running example

$[7, 3, 5, 2, 6, 4] \longrightarrow [2, 3, 4, 5, 6, 7]$

Request	Rank	Answer
Minimum	1	2
Middle order statistics	3, 4	4, 5
Fifth-smallest	5	6
Maximum	6	7

Convention

State whether ranks start at 1 or 0. In this lecture, “ k th smallest” uses ranks 1 through n . For an even number of numerical values, the conventional median often averages the two middle order statistics.

Finding an extreme needs only one scan

```
FUNCTION MINIMUM(A)
```

```
INPUT: non-empty array A
```

```
OUTPUT: smallest value in A
```

```
1   best = A[0]
2   FOR i = 1 TO LENGTH(A) - 1 DO
3       IF A[i] < best THEN
4           best = A[i]
5   RETURN best
```

Exact comparison count

Every value after the first is compared once:

$$T(n) = n - 1.$$

Complexity

Time $\Theta(n)$; auxiliary space $\Theta(1)$.
The maximum algorithm is identical except that $<$ is replaced by $>$.

Strategies for the k th-smallest value

Strategy	Typical time	When useful
Sort, then index	$\Theta(n \log n)$	Many later rank queries or sorted output needed
Repeat minimum selection	$\Theta(kn)$	Very small k , simple implementation
Quickselect	Expected $\Theta(n)$	One rank query on an in-memory array

Algorithm design principle

Solve only the amount of ordering that the required output needs.

Quickselect reuses quicksort's partition

Core idea

Partition around a pivot. The pivot's final index tells us which side can contain the requested rank; discard the other side.



ranks 1–2 rank 3 ranks 4–6

Example

To find the fifth-smallest value, ignore the pivot and the entire left partition; continue only in the right partition.

Trace quickselect for the fifth-smallest value

One possible execution: the exact trace depends on pivot choices.

Active range	Pivot rank	Decision
[7, 3, 5, 2, 6, 4]	4 becomes rank 3	Search ranks 4–6
[7, 6, 5]	5 becomes rank 4	Search ranks 5–6
[7, 6]	6 becomes rank 5	Return 6

What is avoided?

The elements with ranks below the pivot never need further processing.

State to track

The active bounds and the target index, not a fully sorted prefix.

Quickselect pseudocode

```
FUNCTION QUICKSELECT(A, k)
INPUT: array A and rank k,  $1 \leq k \leq \text{LENGTH}(A)$ 
OUTPUT: kth-smallest value
1   target = k - 1
2   low = 0; high = LENGTH(A) - 1
3   WHILE low <= high DO
4       p = PARTITION(A, low, high)
5       IF p == target THEN
6           RETURN A[p]
7       ELSE IF target < p THEN
8           high = p - 1
9       ELSE
10          low = p + 1
```

Invariant

The target value is always inside $A[\text{low}..\text{high}]$.

Side effect

Quickselect rearranges the array but does not completely sort it.

Quickselect complexity

Intuition with balanced partitions

$$n + \frac{n}{2} + \frac{n}{4} + \dots < 2n$$

Only one side is searched after each partition.

Repeated poor pivots

$$n + (n - 1) + (n - 2) + \dots$$

Worst-case running time is $\Theta(n^2)$.

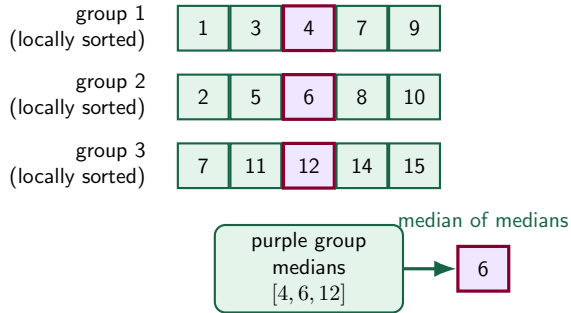
Practical protection

With randomized pivots, Quickselect has expected $\Theta(n)$ time and consistently unbalanced partitions become unlikely.

Median of medians: choosing a reliable pivot

Idea

Instead of choosing an arbitrary quickselect pivot, divide the input into groups of five, find each group median, then select the median of those medians as the pivot.



Important: for a large list of group medians, find their median recursively. The resulting pivot need not be the original array's true median.

Advanced note: median of medians and linear worst case

A guaranteed useful pivot

There are about $n/5$ groups. At least half of their medians are at least the chosen pivot p , and each such group contains three values at least p . The same argument holds on the smaller side.

$$\text{At least } \frac{3n}{10} - O(1)$$

values can be discarded from either side; the remaining subproblem has size at most $7n/10 + O(1)$.

Recurrence and result

$$T(n) \leq T(\lceil n/5 \rceil) + T(7n/10 + O(1)) + cn.$$

The recursive fractions satisfy

$$\frac{1}{5} + \frac{7}{10} = \frac{9}{10} < 1,$$

so the recurrence is $O(n)$. Since selection must inspect the input, it is also $\Omega(n)$; therefore the deterministic worst-case time is

$$\Theta(n).$$

Finding top- k values

Problem

Return the k largest or k smallest values without necessarily sorting all n input values.

Method	Typical time	When useful
Sort all values	$\Theta(n \log n)$	Need complete sorted output
Quickselect, then scan	Expected $\Theta(n)$	Need only the top- k set
Heap of size k	$O(n \log k)$	$k \ll n$ or data arrives as a stream

Key idea

Top- k is between selection and sorting: more than one rank is needed, but a complete total order may still be unnecessary.

Self-check: selection

- ① In $[8, 2, 9, 1, 5, 7]$, what is the fourth-smallest value?
- ② Why does finding the minimum not require sorting?
- ③ When is sorting better than Quickselect for rank queries?
- ④ For $k = 10$ and very large n , why can a heap of size k be useful?
- ⑤ Does Quickselect return a fully sorted array? Explain briefly.

Discussion rule

Give the answer, then justify it using the required amount of ordering.

Activity: choose a selection method

Scenarios

- 1 Find one median in an unsorted array.
- 2 Report the ten largest values after all data is collected.
- 3 Answer thousands of rank queries on fixed data.
- 4 Find only the minimum value.

Student task

Choose among scanning, sorting, quickselect, and a heap. Explain the required time, memory, and whether input modification is acceptable.

SECTION 2

Searching

The searching problem

Input and output

Given a collection and a target key, return a matching position or record; return `NONE` when no match exists.

Questions to specify

- Is the collection ordered?
- Are duplicate keys allowed?
- Return first, last, any, or all matches?

Cost model

Count key comparisons and any preprocessing needed to organize the data.

Linear search: inspect values from left to right

1	2	3	4		
7	3	5	2	6	4

Target 2

The first match occurs after four comparisons. Values after the match are not inspected.

No ordering assumption

Linear search works on arbitrary sequences and linked structures.

Linear-search pseudocode and cases

```
FUNCTION LINEAR_SEARCH(A, target)
INPUT: array A and a target value
OUTPUT: an index of target, or NONE
1   FOR i = 0 TO LENGTH(A) - 1 DO
2       IF A[i] == target THEN
3           RETURN i
4   RETURN NONE
```

Comparisons

Best: 1

Worst: n

Average successful search: about $(n + 1)/2$ when all positions are equally likely

Complexity

Best $\Theta(1)$; average and worst $\Theta(n)$.

Can ordering help us search faster?

Unsorted data

[7, 3, 5, 2, 6, 4]

No position rule tells us where the target might be. In the worst case, every value may need inspection.

Sorted data

[2, 3, 4, 5, 6, 7]

Comparing with the middle value tells us which half cannot contain the target.

Main message

Fast search usually comes from extra structure in the data representation.

Binary search requires sorted data

Precondition

The searched keys must already be ordered. Binary search compares the target with the middle value and discards half of the remaining range.

middle

2	3	4	5	6	7
---	---	---	---	---	---

Target 6

Because $6 > 4$, indices to the left of and including the middle cannot contain the target.

Trace binary search for target 6

Step	low	high	mid	$A[mid]$	Action
1	0	5	2	4	discard indices 0..2
2	3	5	4	6	return index 4

Search interval invariant

If the target exists, at the start of every iteration it lies somewhere in the inclusive interval $[low, high]$.

Progress

Each unsuccessful comparison strictly reduces the interval size.

Iterative binary-search pseudocode

```
FUNCTION BINARY_SEARCH(A, target)
INPUT: sorted array A
OUTPUT: an index of target, or NONE
1   low = 0
2   high = LENGTH(A) - 1
3   WHILE low <= high DO
4       mid = low + (high - low) DIV 2
5       IF A[mid] == target THEN
6           RETURN mid
7       ELSE IF A[mid] < target THEN
8           low = mid + 1
9       ELSE
10          high = mid - 1
11  RETURN NONE
```

Complexity

After t steps the range has size at most $n/2^t$. Therefore the worst case is $\Theta(\log n)$.

Boundary condition

Use $\text{low} \leq \text{high}$; an interval with one element still needs to be checked.

Why binary search is correct

- 1 Initially, every valid index is inside $[low, high]$.
- 2 If $A[mid] < target$, sorted order proves indices $low..mid$ cannot contain the target.
- 3 If $A[mid] > target$, sorted order proves indices $mid..high$ cannot contain the target.
- 4 The interval shrinks after every unsuccessful comparison.
- 5 A match is returned, or the empty interval proves absence.

Important distinction

Binary search is fast only after data is sorted. For one search, sorting an unsorted array first may cost more than a linear scan.

Variant: find the first occurrence

```
1  FUNCTION FIRST_OCCURRENCE(A, target)
2  INPUT: sorted array A and a target value
3  OUTPUT: first index of target, or NONE
4      low = 0; high = LENGTH(A) - 1
5      answer = NONE
6      WHILE low <= high DO
7          mid = low + (high - low) DIV 2
8          IF A[mid] >= target THEN
9              IF A[mid] == target THEN answer = mid
10             high = mid - 1
11          ELSE
12              low = mid + 1
13      RETURN answer
```

Why continue after a match?

A matching value may also occur at a smaller index, so remember the match and search left.

Uses

Counting duplicates, locating ranges, and implementing lower-bound queries.

Searching with Python tools

Sequence search

```
target in values  
values.index(target)
```

These operations use a linear scan for a list.

Ordered search

```
from bisect import bisect_left  
i = bisect_left(values, target)  
found = i < len(values) and values[i] == target
```

Set and dictionary lookup

```
student_id in id_set  
record = records_by_id.get(student_id)
```

Hash-based lookup is expected $O(1)$, but requires extra storage and does not provide sorted-order queries.

Do not choose the algorithm alone

Representation	Typical lookup	Update cost	Best fit
Unsorted list	Linear, $\Theta(n)$	Append often $O(1)$	Few searches, frequent simple inserts
Sorted array/list	Binary, $\Theta(\log n)$	Insert/delete $\Theta(n)$	Many range or ordered queries
Hash set/dict	Expected $O(1)$	Expected $O(1)$	Exact-key membership or retrieval
Balanced search tree	$O(\log n)$	$O(\log n)$	Dynamic ordered data and range queries

Whole-workload thinking

The best search algorithm depends on how the data was organized and how often searches and updates occur.

Common binary-search mistakes

Frequent errors

- Applying it to unsorted data
- Mixing inclusive and exclusive bounds
- Updating to `mid` instead of `mid +/- 1`
- Ignoring duplicate-key requirements
- Returning insertion position as a confirmed match

Testing checklist

Test an empty array, one element, target at each end, absent target, duplicates, and an even-length interval.

Debugging tool

Trace `low`, `mid`, and `high` after every step.

Self-check: searching

- ① Why is binary search invalid on an unsorted list?
- ② In binary search, why should an inclusive interval use `low <= high`?
- ③ What should change if duplicate keys are allowed and the first match is required?
- ④ Which structure is better for exact membership: list, sorted list, or hash set?
- ⑤ Which structure is better for range queries: hash table or sorted structure?

Discussion rule

State the representation first, then state the search cost.

Integrated activity: product catalogue

Situation

A catalogue receives 50 updates per day and serves 100,000 exact product-ID searches plus 2,000 price-range searches per day.

Student task

- 1 Choose representations for exact-ID and price-range queries.
- 2 Explain whether one structure is enough.
- 3 State the expected lookup and update costs.
- 4 Explain how sorting by price can report the ten most expensive products.

Design insight

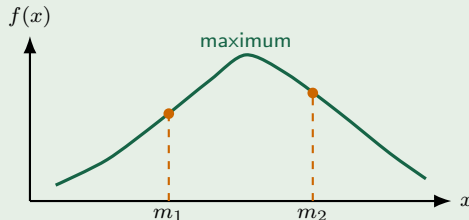
Real systems often maintain more than one index because no single data organization is optimal for every query.

Optional: ternary search on a unimodal function

Core idea

Ternary search can locate the optimum of a **unimodal function**: values increase then decrease, or decrease then increase, with one optimum.

Single-peak shape



Example

Suppose $f(x)$ is profit at price x : profit rises, reaches one peak, then falls. Search for the price that maximizes profit.

One step for a maximum

If $f(m_1) < f(m_2)$, the maximum cannot lie to the left of m_1 ; discard that third. Otherwise, discard the part right of m_2 .

Stopping and complexity

Discrete interval: $\Theta(\log n)$ iterations. Continuous interval: stop when its width is within the required precision.

Ternary-search pseudocode

```
FUNCTION TERNARY_SEARCH_MAX(f, left, right, eps)
INPUT: unimodal function f on [left, right]
OUTPUT: approximate maximizer of f
1   WHILE right - left > eps DO
2       m1 = left + (right - left) / 3
3       m2 = right - (right - left) / 3
4       IF f(m1) < f(m2) THEN
5           left = m1
6       ELSE
7           right = m2
8   RETURN (left + right) / 2
```

For a maximum

If $f(m_1) < f(m_2)$, the peak cannot be in the left third. Otherwise, discard the right third.

Precondition

Ternary search is valid only when the function or sequence is unimodal.

Discrete ternary-search pseudocode

```
FUNCTION TERNARY_SEARCH_MAX_ARRAY(A)
INPUT: unimodal array A
OUTPUT: index of a maximum value
1   low = 0; high = LENGTH(A) - 1
2   WHILE high - low > 3 DO
3       m1 = low + (high - low) DIV 3
4       m2 = high - (high - low) DIV 3
5       IF A[m1] < A[m2] THEN
6           low = m1 + 1
7       ELSE
8           high = m2 - 1
9   best = low
10  FOR i = low + 1 TO high DO
11      IF A[i] > A[best] THEN best = i
12  RETURN best
```

Self-check: ternary search

- ① What property must the input satisfy before ternary search is valid?
- ② For maximizing profit by price, why might binary search not apply directly?
- ③ In the maximum version, what can be discarded when $f(m_1) < f(m_2)$?
- ④ Why does the discrete version scan the final small interval directly?

Key takeaways

Selection

Do not sort more than the output requires. Quickselect can find one order statistic in expected $\Theta(n)$ time; top- k can often avoid full sorting.

Searching

Without useful organization, search may require $\Theta(n)$ comparisons. Sorted order enables binary search in $\Theta(\log n)$.

Data-structure thinking

Search complexity cannot be separated from data organization, update cost, and query workload.

After-class practice

- ① Trace quickselect for the third-smallest value of $[9, 1, 7, 3, 8, 2, 6]$.
- ② Design an algorithm that returns both minimum and maximum.
- ③ Find the five largest values in a list without fully sorting it.
- ④ Trace binary search for present and absent targets in an even-length array.
- ⑤ Modify binary search to find the last occurrence of a duplicated key.
- ⑥ Compare a list, sorted list, dictionary, and set for a student-ID system.

Short programming exercises

- 1 Write Python code for `minimum(values)` without using `min`.
- 2 Write `linear_search(values, target)` and count comparisons.
- 3 Write `binary_search(sorted_values, target)` and print `low`, `mid`, and `high` at each step.
- 4 Use `bisect_left` to test whether a target occurs in a sorted list.
- 5 Implement ternary search for a simple unimodal function, for example $f(x) = -(x - 3)^2 + 10$.